Linear Regression with the California Housing Dataset

---

4. Univariate Linear Regression

Objective

Univariate Linear Regression uses **only one independent variable (feature)** to predict the target.

In this example we use only

Median Income

to predict

Median House Value

Since previous correlation analysis showed that **Median Income has the strongest relationship with house value**, it is a good single predictor.

Step 1: Select Features

```
# Independent variable (X)
X = df[['median_income']]

# Target variable (y)
y = df['median_house_value']
```

Explanation

`X`

Contains the input feature.

```
Median Income
--------------
3.2
5.1
6.8
...
```

`y`

Contains the value we want to predict.

```
House Value
-----------
120000
220000
340000
...
```

Step 2: Split the Dataset

We divide the data into

- 80% Training
- 20% Testing

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Why split the data?

Training data teaches the model.

Testing data evaluates how well the model performs on unseen data.

Output

```
Training samples : 16,346

Testing samples  : 4,087
```

Step 3: Train the Model

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)
```

Explanation

`LinearRegression()`

Creates the model.

`fit()`

Learns the best straight line

$$y = b_0 + b_1 x$$

where

- b_0 = Intercept
- b_1 = Slope

Step 4: View the Equation

```
intercept = model.intercept_
coefficient = model.coef_[0]

print(f"Intercept : {intercept:.2f}")
print(f"Coefficient : {coefficient:.2f}")
```

Output

```
Intercept : 45035.23

Coefficient : 41751.96
```

Therefore,

$$\boxed{HouseValue = 45035.23 + 41751.96 \times MedianIncome}$$

Interpretation

The coefficient is

```
41751.96
```

This means

> For every one-unit increase in Median Income,
> the predicted house value increases by approximately **$41,752**.

Step 5: Make Predictions

```
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)
```

The model now predicts house values for both datasets.

Step 6: Evaluate the Model

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

train_rmse = np.sqrt(mean_squared_error(y_train, train_predictions))
test_rmse = np.sqrt(mean_squared_error(y_test, test_predictions))

train_r2 = r2_score(y_train, train_predictions)
test_r2 = r2_score(y_test, test_predictions)

print(train_rmse)
print(test_rmse)

print(train_r2)
print(test_r2)
```

Output

```
Training RMSE : 83,418

Testing RMSE  : 84,977

Training R² : 0.4743

Testing R²  : 0.4720
```

Understanding RMSE

RMSE measures prediction error.

Lower RMSE is better.

```
Actual Price

|

|         x

|      x

|   x

| x

+---------------------->

Predicted Price
```

The closer predictions are to actual values,
the lower the RMSE.

Understanding R^2 Score

R^2 measures how much variation in house prices is explained by the model.

Range

```
0   ------------------ 1
Worst               Best
```

Here

```
R² = 0.472
```

Meaning

The model explains about **47% of the variation** in house prices using only Median Income.

Step 7: Plot the Regression Line

```
plt.scatter(X_train, y_train, alpha=0.3)

X_sorted = X_train.sort_values("median_income")
y_sorted = model.predict(X_sorted)

plt.plot(
    X_sorted,
    y_sorted,
    color="red",
    linewidth=2
)

plt.xlabel("Median Income")
plt.ylabel("House Value")
plt.title("Univariate Linear Regression")
plt.show()
```

Why sort the values?

Without sorting,

the regression line jumps randomly.

Sorting makes the line smooth and easy to read.

Summary

Using only one feature

```
Median Income
```

we achieved

Metric	Value

RMSE	84,977
R^2	0.472

This model is simple but does not capture all factors affecting house prices.

---

5. Multivariate Linear Regression

Objective

Instead of using one feature,

we use several features together.

Features include

- Median Income
- Housing Age
- Total Rooms
- Bedrooms
- Population
- Households
- Latitude
- Longitude
- Ocean Proximity

This usually produces a better model.

Step 1: Encode Categorical Data

Machine learning models only understand numbers.

The column

```
ocean_proximity
```

contains text.

Example

```
INLAND

NEAR BAY

NEAR OCEAN

ISLAND
```

We convert them into numbers.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

df["ocean_proximity_encoded"] = encoder.fit_transform(
    df["ocean_proximity"]
)
```

Example

Category	Encoded
INLAND	0
NEAR BAY	1
NEAR OCEAN	2
ISLAND	3

Step 2: Select Features

```
features = [
    "median_income",
    "housing_median_age",
    "total_rooms",
    "total_bedrooms",
    "population",
    "households",
    "latitude",
    "longitude",
    "ocean_proximity_encoded"
]

X = df[features]
y = df["median_house_value"]
```

Now

```
X

MedianIncome

HousingAge

Rooms

Bedrooms

Population

...
```

contains **nine predictors**.

Step 3: Split the Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Same 80/20 split as before.

Step 4: Standardize Features

Different features have different scales.

Example

```
Income

2

5

7


Population

300

1500

5000
```

Large numbers dominate smaller ones.

Standardization converts every feature to approximately

Mean = 0

Standard Deviation = 1

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

Step 5: Train the Model

```
model = LinearRegression()

model.fit(X_train, y_train)
```

Now the model learns from all nine features simultaneously.

Step 6: Display Feature Importance

```
coefficients = pd.DataFrame({
    "Feature": features,
    "Coefficient": model.coef_
})

coefficients["Absolute"] = coefficients["Coefficient"].abs()

coefficients.sort_values(
    "Absolute",
    ascending=False,
    inplace=True
)

print(coefficients)
```

Largest coefficients

```
Latitude

Longitude

Median Income

Bedrooms
```

These have the strongest influence on house value.

Step 7: Make Predictions

```
train_predictions = model.predict(X_train)

test_predictions = model.predict(X_test)
```

Step 8: Evaluate the Model

```
train_rmse = np.sqrt(
    mean_squared_error(y_train, train_predictions)
)

test_rmse = np.sqrt(
    mean_squared_error(y_test, test_predictions)
)

train_r2 = r2_score(y_train, train_predictions)

test_r2 = r2_score(y_test, test_predictions)
```

Output

Metric	Value
Training RMSE	69,410
Testing RMSE	70,172
Training R^2	0.636
Testing R^2	0.640

Interpretation

Compared to the previous model

RMSE decreased

```
84,977

↓

70,172
```

Prediction error became much smaller.

R^2 increased

```
47%

↓

64%
```

The model now explains about **64% of house price variation**.

This shows that using multiple features significantly improves prediction accuracy.

---

6. Comparing Both Models

Comparison Table

Metric	Univariate	Multivariate
Training RMSE	83,418	69,410
Testing RMSE	84,977	70,172
Training R^2	0.474	0.636
Testing R^2	0.472	0.640

Which Model is Better?

The **Multivariate Linear Regression** model performs better because:

- It has a lower RMSE, meaning smaller prediction errors.
- It has a higher R^2 score, explaining more variation in house prices.
- It uses information from multiple features instead of relying only on median income.

Residual Analysis

Residuals are the prediction errors.

$$\text{Residual} = \text{Actual Value} - \text{Predicted Value}$$

A good model should have residuals that are:

- Centered around zero.
- Randomly scattered.
- Approximately normally distributed.

Common residual plots include:

1. Residuals vs Predicted Values
2. Histogram of Residuals
3. Q-Q Plot

These plots help determine whether the assumptions of linear regression are reasonably satisfied.

Final Conclusion

Model	R^2	RMSE	Recommendation
Univariate	0.472	84,977	Good for learning linear regression concepts.
Multivariate	0.640	70,172	Recommended for making more accurate predictions.

The multivariate model clearly outperforms the univariate model because it leverages multiple relevant features to explain house prices.